Artificial Intelligence: Principles and Techniques

Mini Projects based on Stanford CS221 (Spring 2026) coursework

Theoretical Part

Written solutions to Assignments 1–7

Group 13

#	Full name	Student Number	Registration Number
1	MUJUNI INNOCENT	2400707155	24/U/07155/PS
2	**[TODO: name]**	**[TODO:]**	**[TODO:]**
3	**[TODO: name]**	**[TODO:]**	**[TODO:]**
4	**[TODO: name]**	**[TODO:]**	**[TODO:]**
5	**[TODO: name]**	**[TODO:]**	**[TODO:]**
6	**[TODO: name]**	**[TODO:]**	**[TODO:]**
7	**[TODO: name]**	**[TODO:]**	**[TODO:]**
8	**[TODO: name]**	**[TODO:]**	**[TODO:]**
9	**[TODO: name]**	**[TODO:]**	**[TODO:]**
10	**[TODO: name]**	**[TODO:]**	**[TODO:]**

The programming parts are in the companion notebook `CS221_Mini_Projects.ipynb`, which uses the same numbering.

October 27, 2026

Contents

Assignment 1: Foundations **2**
Problem 1: Optimization and probability . . . 3
Problem 2: Complexity . . . 4
Problem 3: Ethical issue spotting . . . 5
Problem 4: Programming . . . 5

Assignment 2: Sentiment Classification **5**
Problem 1: Building intuition . . . 6
Problem 2: Predicting movie ratings . . . 6
Problem 3: Sentiment classification . . . 7
Problem 4: Toxicity classification and maximum group loss . . . 7
Problem 5: K-means clustering . . . 9

Assignment 3: Route Planning **10**
Problem 1: Grid city . . . 11
Problem 2: Finding shortest paths . . . 11
Problem 3: Finding shortest paths with unordered waypoints . . . 12
Problem 4: Speeding up search with A* . . . 13

Assignment 4: Controlling MountainCar **13**
Problem 1: Value iteration . . . 14
Problem 2: Transforming MDPs . . . 14
Problem 3: Value iteration on Mountain Car . . . 15
Problem 4: Q-learning Mountain Car . . . 15
Problem 5: Safe exploration . . . 15

Assignment 5: Pac-Man **16**
Problem 1: Minimax . . . 17
Problem 2: Alpha-beta pruning . . . 17
Problem 3: Expectimax . . . 17
Problem 4: Evaluation function (extra credit) . . . 18
Problem 5: AI (mis)alignment and reward hacking . . . 18

Assignment 6: Bayesian Networks (Car Tracking) **19**
Problem 1: Emission probabilities . . . 20
Problem 2: Transition probabilities . . . 20
Problem 3: Which car is it? . . . 21
Problem 4: Ethics in advanced technologies . . . 22

Assignment 7: From Language to Logic **23**

Problem 1: Propositional logic. 24

Problem 2: First-order logic 24

Problem 3: Liar puzzle. 24

Problem 4: Odd and even integers 24

Problem 5: Semantic parsing (extra credit) 25

Problem 6: Explainability of logic-based systems 25

Problem 7: Applications of soundness and completeness 25

Assignment 1: Foundations

Problem 1: Optimization and probability

(1a) Weighted quadratic minimisation

Let $x_1, \ldots, x_n \in \mathbb{R}$ and weights $w_1, \ldots, w_n > 0$. For $f(\theta) = \sum_{i=1}^{n} w_i(\theta - x_i)^2$ (θ scalar), find the minimiser, show it is a minimum, and explain what goes wrong if some w_i are negative.

What we expect: Expression for the minimising θ and its derivation; a short argument that it is a minimum; 1–2 sentences on negative weights.

Solution. [TODO: answer]

(1b) Swapping min and sum

Let $f(\mathbf{x}) = \min_{s\in[-1,1]} \sum_{i=1}^{d} s\, x_i$ and $g(\mathbf{x}) = \sum_{i=1}^{d} \min_{s_i\in[-1,1]} s_i x_i$ for $\mathbf{x} \in \mathbb{R}^d$. Which of $f \le g$, $f = g$, $f \ge g$ holds for all $\mathbf{x}$? Prove it.

What we expect: A short (3–5 line) proof using mathematical notation.

Solution. [TODO: answer]

(1c) Expected points of a dice game

Roll a fair six-sided die until a 1 appears. Each 3 wins a points, each 6 loses b points, and $2, 4, 5$ change nothing. What is the expected number of points at the end, as a function of a, b?

What we expect: A recurrence for the value V, a short explanation of it, and the solved expression.

Solution. [TODO: answer]

(1d) Maximum likelihood for a coin

A coin with $P(\mathrm{H}) = p$, $0 < p < 1$, gives $\{\mathrm{T}, \mathrm{H}, \mathrm{H}, \mathrm{H}, \mathrm{H}\}$, so $L(p) = p^4(1-p)$. Which p maximises L? Prove it is a maximum and give an intuitive interpretation.

What we expect: The maximiser with its derivation (e.g. via $\log L$), proof that it is a maximum, and a 1-sentence interpretation.

Solution. [TODO: answer]

(1e) Conditional probability

Events A, B satisfy $P(A \mid B) = P(B \mid A)$, $P(A \cup B) = \frac{1}{3}$ and $P(A \cap B) > 0$. Prove that $P(A) > \frac{1}{6}$.

What we expect: A short (∼5 line) proof.

Solution. [TODO: answer]

(1f) Gradient

For $\mathbf{w} \in \mathbb{R}^d$, constants $\mathbf{a}_i, \mathbf{b}_j \in \mathbb{R}^d$, $\lambda \in \mathbb{R}$ and $n \in \mathbb{N}$,

$$f(\mathbf{w}) = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} (\mathbf{a}_i^\top \mathbf{w} - \mathbf{b}_j^\top \mathbf{w})^2 \right) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2.$$

Compute $\nabla f(\mathbf{w})$ (no summations over dimensions).

What we expect: A compact expression for the gradient and its derivation (∼5 lines).

Solution. [TODO: answer]

Problem 2: Complexity

(2a) Placing three rectangles

On an $n \times n$ grid of points, how many ways are there to place 3 axis-aligned rectangles whose corners are grid points (degenerate and overlapping rectangles allowed)? Answer as $O(n^c)$ or $O(c^n)$.

What we expect: A big-O bound and ∼2 sentences of reasoning.

Solution. [TODO: answer]

(2b) Minimum-cost monotone path

On an $n \times 2n$ grid, go from $(1, 1)$ to $(n, 2n)$ using only down/right steps. Each point has cost $c(i, j)$ (possibly negative, $O(1)$ to evaluate), and a path's cost is the sum over the points it visits. Give the most efficient algorithm for the minimum path cost and its runtime.

What we expect: Algorithm description (∼5 sentences), the big-O runtime, and why.

Solution. [TODO: answer]

Problem 3: Ethical issue spotting

For each scenario, either identify one concern from the NeurIPS Ethics Guidelines (General Ethical Conduct / Potential Negative Social Impacts), copy it out and underline it, and justify it, or argue that there is no concern.

What we expect: A 2–5 sentence paragraph per scenario.

(3a) Loan-default prediction

A loan-default model relies mainly on location, accepts urban and rejects rural applicants, and has a significantly higher false-positive rate for Black and male applicants.

Solution. [TODO: answer]

(3b) Stylometry

Highly accurate ML stylometry predicts the authors of anonymous texts (including code) and could de-anonymise a wide range of writing.

Solution. [TODO: answer]

(3c) Celebrity face dataset

Millions of celebrity faces were scraped from Google Images, without permission and including copyrighted images, to build facial recognition; copyrighted images are distributed as URLs plus bounding boxes.

Solution. [TODO: answer]

(3d) Plant-species identification

A model identifies plant species from a photo. It was trained on iNaturalist photos whose users consented to research use, and it is only used inside the app.

Solution. [TODO: answer]

Problem 4: Programming

*Programming part. The implementation, documentation and test output are in the notebook, section **1.4a–1.4f**.*

Functions: `find_alphabetically_first_word`, `euclidean_distance`, `mutate_sentences`, `sparse_vector_dot_product`, `increment_sparse_vector`, `find_nonsingleton_words`.

Assignment 2: Sentiment Classification

Problem 1: Building intuition

Mini-reviews: (-1) *not good*; (-1) *pretty bad*; $(+1)$ *good plot*; $(+1)$ *pretty scenery*. $\phi(x)$ = word counts and $\text{Loss}_{\text{hinge}}(x, y, \mathbf{w}) = \max\{0,\, 1 - \mathbf{w} \cdot \phi(x)\, y\}$.

(1a) Four SGD steps

Run one SGD update per review, in the order given, with $\eta = 0.1$ from $\mathbf{w} = \mathbf{0}$ (gradient $= 0$ when the margin is exactly 1). Give the final weights of (*pretty, good, bad, plot, not, scenery*).

What we expect: The weight vector in that order, e.g. $[0.1, 0.2, 0.3, 0.4, 0.5, 0.6]$.

Solution. **[TODO: show each update, then the final vector]**

(1b) A non-separable dataset

Dataset: (-1) *bad*; $(+1)$ *good*; $(+1)$ *not bad*; (-1) *not good*. Prove that no linear classifier $f_{\mathbf{w}}(x) = \text{sign}(\mathbf{w} \cdot \phi(x))$ on word-count features achieves zero error, then propose one extra feature that fixes this.

What we expect: A short proof (∼3–5 sentences) and a viable additional feature.

Solution. **[TODO: answer]**

Problem 2: Predicting movie ratings

Predictor $\sigma(\mathbf{w} \cdot \phi(x))$ with $\sigma(z) = (1 + e^{-z})^{-1}$, rating $y \in [0, 1]$. (No Wolfram Alpha or similar.)

(2a) Squared loss

Write $\text{Loss}(x, y, \mathbf{w})$ for squared loss on a single data point.

What we expect: A mathematical expression (may use σ).

Solution. **[TODO: answer]**

(2b) Its gradient

Compute $\nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w})$ in terms of $p = \sigma(\mathbf{w} \cdot \phi(x))$.

What we expect: A mathematical expression for the gradient.

Solution. [TODO: answer]

(2c) Vanishing gradients

For one data point with $y = 1$, under what conditions on $\mathbf{w}$ does $\|\nabla_{\mathbf{w}}\text{Loss}\|_2$ become arbitrarily small? Can it be exactly zero ($\|\mathbf{w}\|_2$ large but finite)?

What we expect: 1–2 sentences on the conditions; 1–2 sentences on whether it can be exactly zero.

Solution. [TODO: answer]

Problem 3: Sentiment classification

*Programming part. The implementation, documentation and test output are in the notebook, section **2.3a–2.3d**.*

`extract_word_features`, `learn_predictor` (SGD on hinge loss; train error $< 4\%$, validation error $< 30\%$), `generate_dataset`, `extract_character_features`.

(3e) Character n-grams

Run the linear predictor with `extract_character_features` for several n. Which n gives the smallest validation error, and why is it nearly as good as word features? Construct a one-sentence review for which character n-grams probably beat word features, and explain why.

What we expect: A paragraph (∼4–6 sentences) with the best n and the reason; a one-sentence review with an explanation.

Solution. [TODO: table of validation error vs. n (from notebook §2.3e) and discussion]

Problem 4: Toxicity classification and maximum group loss

$\phi(x) = [1, d, t]$: bias, demographic mention $d \in \{0, 1\}$, toxic words $t \in \{0, 1\}$; label $y \in \{-1, 1\}$ (1 = toxic). Groups are defined by (y, d). Zero-one loss; $\text{TrainLoss}_{\max}(\mathbf{w}) = \max_g \text{TrainLoss}_g(\mathbf{w})$.

y	d	t	# points
−1	0	0	63
−1	0	1	27
−1	1	0	7
−1	1	1	3
1	0	0	3
1	0	1	7
1	1	0	27
1	1	1	63
	Total		200

Classifier D: $\mathbf{w} = [-0.1, 1, 0]$ Classifier T: $\mathbf{w} = [-0.1, 0, 1]$

(4a) Behaviour of D and T

Describe in words how each classifier behaves.

What we expect: For each classifier, an "if and only if" statement for when $f_{\mathbf{w}}(x) = 1$.

Solution. [TODO: answer]

(4b) Losses of classifier D

Compute D's average loss, its per-group losses, and its maximum group loss.

What we expect: The average loss, the completed table below, and the maximum group loss.

Solution.

	$y = 1$	$y = -1$
$d = 1$	[TODO:]	[TODO:]
$d = 0$	[TODO:]	[TODO:]

Average loss: [TODO:]. Maximum group loss: [TODO:].

(4c) Losses of classifier T; comparison

Compute the same three quantities for T. Which classifier has the lower average loss, and which has the lower maximum group loss?

What we expect: The average loss, completed table, and maximum group loss, plus both comparisons.

Solution.

	$y = 1$	$y = -1$
$d = 1$	[TODO:]	[TODO:]
$d = 0$	[TODO:]	[TODO:]

Average loss: [TODO:]. Maximum group loss: [TODO:]. Comparison: [TODO:].

(4d) Fairness and the mixed objective

$\text{TrainLoss}_{\text{new}}(\mathbf{w}) = \lambda\,\text{TrainLoss}_{\text{avg}}(\mathbf{w}) + (1-\lambda)\,\text{TrainLoss}_{\max}(\mathbf{w})$, with $\lambda \in [0, 1]$. Use the ethical frameworks from the *Algorithms and Distribution* module.

(i) $\lambda = 1$: describe the optimal classifier and the framework it aligns with.

What we expect: 1–2 sentences, justified by the framework.

Solution. [TODO: answer]

(ii) $\lambda = 0$: same question.

What we expect: 1–2 sentences, justified by the framework.

Solution. [TODO: answer]

(iii) $\lambda = 0$ with a new training set in which group $(y = 1, d = 1)$ has size 1 (the other groups have 65, 65, 69). How would you factor group size into $\text{TrainLoss}_{\max}$?

What we expect: The adjustment, justified in 1–2 sentences.

Solution. [TODO: answer]

(iv) What λ would you deploy on a real platform that cares equally about average and maximum group loss? Discuss the trade-offs with reference to the frameworks.

What we expect: 3–5 sentences.

Solution. [TODO: answer]

Problem 5: K-means clustering

$\phi(x_1) = [0, 0]$, $\phi(x_2) = [4, 0]$, $\phi(x_3) = [6, 0]$, $\phi(x_4) = [11, 0]$.

(5a) 2-means by hand

Run 2-means to convergence from (i) $\mu_1 = [0, 0]$, $\mu_2 = [11, 0]$ and (ii) $\mu_1 = [0, 0]$, $\mu_2 = [4, 0]$. Give the assignments z, centres μ and final total loss.

What we expect: For each initialisation: centres and assignments at every step, and the final loss.

Solution. [TODO: answer for both initialisations]

(5b) Implementing k-means

*Programming part. The implementation, documentation and test output are in the notebook, section **2.5b**.*

(5c) Scaling invariance

If every dimension of the data and the initial centroids is scaled by the same non-zero factor, are the same clusters guaranteed? What if only some dimensions are scaled?

What we expect: For each sub-question: a short explanation if the answer is yes, or a concrete numerical counterexample if no.

Solution. **[TODO: answer]**

Assignment 3: Route Planning

Problem 1: Grid city

Infinite grid of integer locations (x, y), with $m, n \geq 0$:

- $s_{\text{start}} = (0, 0)$, $\quad \text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$
- $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$, $\quad \text{Succ}(s, a) = s + a$
- $\text{Cost}((x, y), a) = 1 + \max(x, 0)$

(1a) Minimum cost

What is the minimum cost of reaching (m, n) from $(0, 0)$? Describe a path achieving it. Is it unique?

What we expect: An expression for the minimum cost; 1–2 sentences describing one optimal path and whether it is unique.

Solution. [TODO: answer]

(1b) UCS on this problem

True or false: (i) UCS never terminates because there are infinitely many states. (ii) UCS returns the minimum-cost path and explores only (x, y) with $0 \leq x \leq m$, $0 \leq y \leq n$. (iii) UCS returns the minimum-cost path and explores only locations whose past cost is $\leq$ the minimum cost to (m, n).

What we expect: T/F for each sub-part.

Solution. (i) [TODO:] (ii) [TODO:] (iii) [TODO:]

(1c) UCS on an arbitrary graph

True or false: (i) Adding a connection between two nodes cannot increase the minimum cost between any start and end node. (ii) Making an action's cost small enough (possibly negative) guarantees that it appears in the path UCS returns, if a path exists. (iii) Increasing every action's cost by 1 does not change the minimum-cost path, only its cost.

What we expect: T/F for each sub-part.

Solution. (i) [TODO:] (ii) [TODO:] (iii) [TODO:]

Problem 2: Finding shortest paths

(2a) `ShortestPathProblem`

*Programming part. The implementation, documentation and test output are in the notebook, section **3.2a**.*

(2b) Stanford shortest-path experiments

Choose start locations and end tags with `get_stanford_shortest_path_problem`, and visualise them. Give (i) a setting that gave new insight into travelling around campus (was the system useful?) and (ii) a setting where the minimum-cost path is *not* desirable. Is that caused by modelling assumptions (missing tags or paths, inaccurate locations or distances)?

What we expect: Screenshots of both visualisations; one or more sentences on something learned about travelling, and something incorrect about the map or the modelling assumptions.

Solution. [TODO: screenshots: figures/03_route/2b_*.png, plus discussion]

(2c) Negative externalities

What negative externalities could follow from wide deployment of this shortest-path system? Discuss the effects on (1) users and (2) non-users, and ways to reduce the mismatch between the model and the real world (see "Why Traffic Apps Make Congestion Worse", "The Perfect Selfishness of Mapping Apps", and the Externalities module).

What we expect: 3–5 sentences: two externalities (one for users, one for non-users) and at least one mitigation.

Solution. [TODO: answer]

Problem 3: Finding shortest paths with unordered waypoints

(3a) `WaypointsShortestPathProblem`

*Programming part. The implementation, documentation and test output are in the notebook, section **3.3a**.*

(3b) Size of the state space

With n locations and k waypoint tags, what is the maximum number of states UCS could visit under a suitable state definition for 3a?

What we expect: An expression in n and k with a brief justification.

Solution. [TODO: answer]

(3c) Stanford waypoints experiment

Choose a start location, waypoint tags (at least two) and an end tag with `get_stanford_waypoints_shortest_path_problem`, and visualise the route. What is unexpected or interesting about it? Does it match your expectations, and what important factors does it fail to capture?

What we expect: A screenshot, the list of waypoint tags, and one or more sentences of discussion.

Solution. **[TODO: screenshot figures/03_route/3c_*.png, tags used, discussion]**

(3d) Ride-sharing and dual use

Ride-sharing apps use similar routing and have been criticised for exploitative nudges (e.g. "forward dispatch"). How could unordered waypoints improve the driver experience between a drop-off and the next pick-up, and what driver information would you need? As a dual-use concern, how could a company use the feature to raise profits at the expense of driver health, or what are the downsides of collecting that information?

What we expect: 3–5 sentences: (a) waypoints that improve the driver experience; (b) at least one dimension of *driver identity* (not trip attributes such as time of day) that informs the choice; (c) a harmful use or a downside of the data collection.

Solution. **[TODO: answer]**

Problem 4: Speeding up search with A*

*Programming part. The implementation, documentation and test output are in the notebook, section **3.4a–3.4c**.*

- `a_star_reduction`: A* as UCS on modified costs
- `StraightLineHeuristic`
- `NoWaypointsHeuristic`

Assignment 4: Controlling MountainCar

Problem 1: Value iteration

States $= \{-2, -1, 0, +1, +2\}$, start state 0, end states ± 2, Actions $= \{a_1, a_2\}$, $\gamma = 1$. For $i \in \{-1, 0, 1\}$: $T(i{-}1 \mid i, a_1) = 0.8$, $T(i{+}1 \mid i, a_1) = 0.2$; $T(i{-}1 \mid i, a_2) = 0.7$, $T(i{+}1 \mid i, a_2) = 0.3$. Reward$(s, a, s') = 10$ if $s' = -2$; 50 if $s' = +2$; -5 otherwise.

(1a) Two iterations

Give $V_i^\star(s)$ for every s after iterations $i = 1, 2$ (with $V_0^\star \equiv 0$, and $V_i^\star = 0$ at end states).

What we expect: All 10 values.

Solution.

	$s=-2$	$s=-1$	$s=0$	$s=+1$	$s=+2$
$V_1^\star$	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
$V_2^\star$	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

[TODO: show the Q-value computations]

(1b) Optimal policy

Using $V_2^\star$, what is $\pi^\star(s)$ for each non-end state?

What we expect: $\pi^\star(s)$ for $s \in \{-1, 0, +1\}$.

Solution. [TODO: answer]

Problem 2: Transforming MDPs

(2a) Removing the discount

Given an MDP with discount $\gamma < 1$ and a solver that only handles $\gamma' = 1$, define a new MDP with States$' =$ States $\cup \{o\}$, the same actions, and $\gamma' = 1$. Specify $T'(s' \mid s, a)$ and Reward$'(s, a, s')$ so that $V_{\text{opt}}(s)$ is unchanged for every $s \in$ States.

What we expect: T' and Reward$'$ as formulas, followed by a short verification that the optimal-value recurrences coincide. Use the notation of the question.

Solution. [TODO: answer]

Problem 3: Value iteration on Mountain Car

*Programming part. The implementation, documentation and test output are in the notebook, section **4.3a–4.3b**.*

`value_iteration` and `ModelBasedMonteCarlo`.

(3c) Model-based value iteration: results

Run `python train.py -agent value-iteration` (three trials). Comment on the reward plots, and discuss when model-based value iteration can perform poorly in general.

What we expect: The plots, plus 2–3 sentences describing them and when the method may fail.

Solution. **[TODO: include figures/04_mountaincar/mcvi_*.png and discuss]**

Problem 4: Q-learning Mountain Car

*Programming part. The implementation, documentation and test output are in the notebook, section **4.4a–4.4c**.*

`TabularQLearning`, `fourier_feature_extractor`, `FunctionApproxQLearning`.

(4d) Tabular vs. function approximation

Run `train.py -agent tabular` and `-agent function-approximation`. Compare the plots. Tabular Q-learning is expected to do better on this task. Why might that be?

What we expect: The plots, plus 2–3 sentences comparing the two and giving possible reasons.

Solution. **[TODO: include figures/04_mountaincar/tabular_*.png, fapprox_*.png and discuss]**

(4e) When function approximation wins

Why could function-approximation Q-learning beat tabular Q-learning in other environments, e.g. with limited exploration, very high-dimensional state spaces, or tight memory?

What we expect: 2–3 sentences.

Solution. **[TODO: answer]**

Problem 5: Safe exploration

(5a) `max_speed` in Gym

Explain how `self.max_speed` is used in Gym's `mountain_car.py`.

What we expect: One sentence.

Solution. [TODO: answer]

(5b) Removing the constraint

Run `train.py -agent function-approximation -max_speed=100000`. Why does the behaviour of the learned policy barely change compared with `max_speed` = 0.07?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(5c) Constrained Q-learning

After implementing `ConstrainedQLearning`, so that only actions with $\text{velocity}_{t+1} < \text{threshold}$ are allowed, with $\text{velocity}_{t+1} = \text{velocity}_t + (\text{action} - 1) \cdot \text{force} - \cos(3\,\text{position}_t) \cdot \text{gravity}$, run `grader.py 5c-helper`. Why do the policies for `max_speed` = 0.065 and an unconstrained `max_speed` differ?

What we expect: 1–2 sentences (only the written answer is graded). Code: notebook §4.5c.

Solution. [TODO: answer]

(5d) Designing safe MDPs for autonomous driving

An autonomous car is trained with RL on public residential roads (pedestrians, vehicles, traffic signals; camera sensing). Design **MDP A** with an exploration policy that could cause harm, and **MDP B**, a modification that reduces or avoids that harm.

What we expect: For each MDP: States, Actions(s), Reward(s, a, s'), an exploration policy, and a 1–2 sentence ethical explanation.

Solution. **MDP A.** [TODO: States / Actions(s) / Reward(s,a,s') / exploration policy / harm]

MDP B. [TODO: States / Actions(s) / Reward(s,a,s') / exploration policy / mitigation]

Assignment 5: Pac-Man

Problem 1: Minimax

(1a) Multi-ghost minimax recurrence

There are $n+1$ agents $a_0, \ldots, a_n$: Pac-Man (a_0, max) and n ghosts (min). One unit of depth means all $n+1$ agents move once. d starts at $d_{\max}$ at the root and decreases. Write $V_{\text{minmax}}(s, d)$ as a piecewise function of IsEnd(s), Utility(s), Eval(s), Player(s), Actions(s) and Succ(s, a); n may be used, $d_{\max}$ may not.

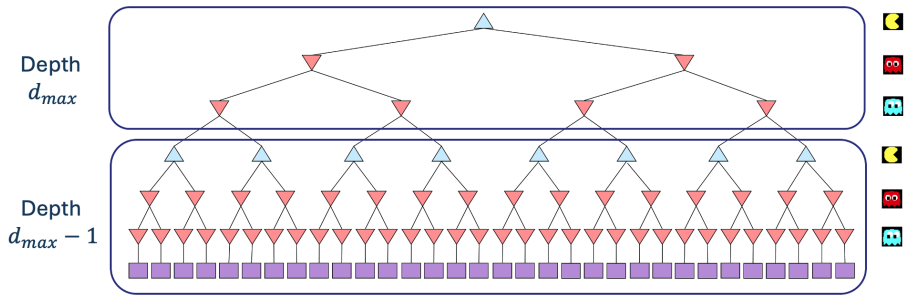


Figure 1: Depth vs. tree height (from the handout).

What we expect: The recurrence as a piecewise function, using exactly these functions.

Solution.

$$V_{\text{minmax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{IsEnd}(s) \\ \text{[TODO:]} & \text{[TODO:]} \end{cases}$$

(1b) `MinimaxAgent`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.1b**.*

Problem 2: Alpha-beta pruning

*Programming part. The implementation, documentation and test output are in the notebook, section **5.2a**.*

Problem 3: Expectimax

(3a) Expectimax recurrence

Write $V_{\text{exptmax}}(s, d)$, the maximum expected utility against ghosts that each pick a legal move uniformly at random, using the same functions as in 1a.

What we expect: A piecewise recurrence in the style of 1a.

Solution. [TODO: answer]

(3b) `ExpectimaxAgent`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.3b**.*

Problem 4: Evaluation function (extra credit)

(4a) `better_evaluation_function`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.4a**.*

(4b) Describing the evaluation function

Describe the evaluation function and its high-level motivation. What else did you try, and what worked and what did not? State whether you consent to the solution being shared if it places in the top 3.

What we expect: A short paragraph.

Solution. [TODO: answer]

Problem 5: AI (mis)alignment and reward hacking

Compare `python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3` with the `ExpectimaxAgent` on the same layout (run each several times).

(5a) Why the agents differ

Why does the minimax agent always rush towards the *closest* ghost, while the expectimax agent (occasionally) does not?

What we expect: One sentence for minimax, stating the assumption that causes this, and one sentence for expectimax.

Solution. [TODO: answer]

(5b) Fixing the objective

Suggest a change to Eval(s) (`score_evaluation_function`) and/or Utility(s) (the final score), or to how the game score is computed, that would stop minimax from dying instantly on `trappedClassic` and make it behave more like expectimax.

What we expect: 1–2 sentences describing the change and why it works.

Solution. [TODO: answer]

(5c) Another real-world example

Describe a realistic scenario outside Pac-Man in which a designer's objective is open to reward hacking or causes negative side effects.

What we expect: 2–5 sentences, stating whether it is reward hacking, a negative side effect, or both, with a justification.

Solution. [TODO: answer]

Assignment 6: Bayesian Networks (Car Tracking)

Set-up: our car's position a_t is observed. The other car's location is C_t (hidden). The microphone reading is $D_t \sim \mathcal{N}(\|a_t - C_t\|_2, \sigma^2)$. Lower-case $p(\cdot)$ denotes local conditional distributions of the Bayesian network; upper-case $\mathbb{P}(\cdot)$ denotes joint or posterior quantities obtained by inference.

Problem 1: Emission probabilities

The other car is stationary: $C_t = C$ for all t.

(1a) Bayesian network

Draw the Bayesian network over $C, D_1, D_2, D_3, a_1, a_2, a_3$, shading the variables observed after $t = 3$.

What we expect: A legible drawing with one arrow from each parent of each variable.

Solution. **[TODO: draw with TikZ: nodes use the styles latent / obs, arrows use edge]**

(1b) Joint distribution

Express $\mathbb{P}(C = c, D_1 = d_1, D_2 = d_2, D_3 = d_3)$ using only local conditional distributions (a_1, a_2, a_3 are known with probability 1).

What we expect: A formula that uses only local conditional distributions.

Solution. **[TODO: answer]**

(1c) `ExactInference.observe`

*Programming part. The implementation, documentation and test output are in the notebook, section **6.1c**.*

Problem 2: Transition probabilities

The other car moves according to $p(c_{t+1} \mid c_t)$, given in code as `self.transProb`.

(2a) Bayesian network

Draw the Bayesian network over $C_1, C_2, C_3, D_1, D_2, D_3, a_1, a_2, a_3$, shading the variables observed after $t = 3$.

What we expect: A legible drawing with one arrow from each parent of each variable.

Solution. [TODO: TikZ drawing]

(2b) Joint distribution

Express $\mathbb{P}(C_1 = c_1, C_2 = c_2, C_3 = c_3, D_1 = d_1, D_2 = d_2, D_3 = d_3)$ using only local conditional distributions.

What we expect: A formula that uses only local conditional distributions.

Solution. [TODO: answer]

(2c) `ExactInference.elapseTime`

*Programming part. The implementation, documentation and test output are in the notebook, section **6.2c**.*

Problem 3: Which car is it?

With K cars, we observe only $\mathbf{E}_t = [E_{t1}, \ldots, E_{tK}]$, a uniformly random permutation of the true noisy distances $\mathbf{D}_t = [D_{t1}, \ldots, D_{tK}]$, so a reading cannot be attributed to a specific car. All distances are distinct.

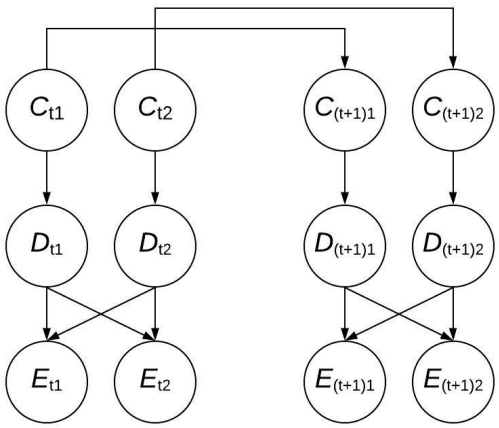


Figure 2: Flow of information for $K = 2$ (from the handout; not a Bayesian network).

(3a) Posterior for $K = 2$, $T = 1$

Write $p(C_{11} = c_{11}, C_{12} = c_{12} \mid \mathbf{E}_1 = \mathbf{e}_1)$ in terms of the Gaussian pdf $p_{\mathcal{N}}(v; \mu, \sigma^2)$ and the priors $p(c_{11}), p(c_{12})$, without D_{11}, D_{12}. (For $K = 1$: $p(c_{11} \mid \mathbf{e}_1) \propto p(c_{11})\, p_{\mathcal{N}}(e_{11}; \|a_1 - c_{11}\|_2, \sigma^2)$.)

What we expect: The expression (proportionality is fine) and the steps taken to derive it.

Solution. [TODO: answer]

(3b) At least $K!$ maximisers

If the prior $p(c_{1i})$ is the same for every car, show that at least $K!$ assignments $(c_{11}, \ldots, c_{1K})$ attain the maximum of $p(C_{11} = c_{11}, \ldots, C_{1K} = c_{1K} \mid \mathbf{E}_1 = \mathbf{e}_1)$ (assume the maximising locations are distinct).

What we expect: A short mathematical argument or a concise explanation.

Solution. [TODO: answer]

Problem 4: Ethics in advanced technologies

(4a) Camera data misuse (dual use)

Self-driving cars often use cameras, which capture faces, licence plates and more. How might camera data or its readings be misused, and what measures could pre-empt the misuse (or why are there none)?

What we expect: 4–6 sentences.

Solution. [TODO: answer]

(4b) Sensor deception

*Programming part. The implementation, documentation and test output are in the notebook, section **6.4b**.*

Observed distance after the attack: $D'_t = \frac{1}{1+\text{skewness}^2} D_t + \sqrt{\frac{2}{1+\text{skewness}^2}}$, implemented in `ExactInferenceWithSensorDeception.observe`.

(4c) Guarding against sensor deception

How could you guard against sensor deception if the type of attack is known in advance? (Hint: sensor redundancy.)

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(4d) Particle filtering vs. exact inference

Run `python drive.py -a -d -k 3 -i particleFilter`. How does the simulation differ from exact inference, and why?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(4e) Privacy–accuracy trade-off

Particle filtering can run on-device (more private and secure) but is less accurate than cloud-based exact inference. How would you weigh this trade-off?

What we expect: 3–5 sentences.

Solution. [TODO: answer]

(4f) Adversarial attack on camera-based training

A camera-based autonomous vehicle is further trained on customers' anonymised camera data. Name an adversarial attack, its potential impact on the vehicle, and a mitigation.

What we expect: 3–5 sentences.

Solution. [TODO: answer]

Assignment 7: From Language to Logic

Problems 1–5 are programming problems (formulas written with `logic.py`). We list the formulas in mathematical notation so the report can be read on its own; the code and grader output are in the notebook.

Problem 1: Propositional logic

*Programming part. The implementation, documentation and test output are in the notebook, section **7.1**.*

(1a) "If it's summer and we're in California, then it doesn't rain." **[TODO: formula]**

(1b) "It's wet if and only if it is raining or the sprinklers are on." **[TODO: formula]**

(1c) "Either it's day or night (but not both)." **[TODO: formula]**

Problem 2: First-order logic

*Programming part. The implementation, documentation and test output are in the notebook, section **7.2**.*

(2a) "Every person has a parent." **[TODO: formula]**

(2b) "At least one person has no children." **[TODO: formula]**

(2c) Define Father(x, y) in terms of Male and Parent. **[TODO: formula]**

(2d) Define Granddaughter(x, y) in terms of Female and Child. **[TODO: formula]**

Problem 3: Liar puzzle

*Programming part. The implementation, documentation and test output are in the notebook, section **7.3a**.*

Suspects: John, Susan, Mark, Nicole. Mark: "It wasn't me!" John: "It was Nicole!" Nicole: "It wasn't me, it was Susan!" Susan: "Nicole's a liar." Exactly one person tells the truth, and exactly one person crashed the server.

[TODO: the six formulas and the answer returned by `grader.py 3a-run`]

Problem 4: Odd and even integers

*Programming part. The implementation, documentation and test output are in the notebook, section **7.4a**.*

[TODO: the six axioms in mathematical notation and the result of `grader.py 4a-run`]

Problem 5: Semantic parsing (extra credit)

*Programming part. The implementation, documentation and test output are in the notebook, section **7.5**.*

Grammar rules for: *every $Noun $Verb some $Noun*; *there is some $Noun that every $Noun $Verb*; *if a $Noun $Verb a $Noun then the former $Verb the latter.*

Problem 6: Explainability of logic-based systems

The four considerations: *respect*; *assessing the fairness of rules*; *contesting and correcting decisions*; *ability to change user behaviour*. Context: the liar puzzle of Problem 3, where a conclusion could have real consequences (e.g. someone being fired).

(6a) What counts as an explanation

Besides the final answer, what does a general first-order logic system, like the one used in Problem 3, return that could count as an explanation?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(6b) Why it may not be adequate

Why might this explanation not be adequate? (Hint: model misspecification, or how the question is framed.)

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(6c) Which considerations are met

Are any of the four explainability considerations met by the explanations a logic-based system produces?

What we expect: 2–4 sentences.

Solution. [TODO: answer]

Problem 7: Applications of soundness and completeness

Sound: $\{f : KB \vdash f\} \subseteq \{f : KB \models f\}$. Complete: $\{f : KB \vdash f\} \supseteq \{f : KB \models f\}$.

(7a) Is an LLM sound or complete?

Treat whatever GPT-3.5 generates as what it "derives". Given that LLMs hallucinate, is GPT-3.5 sound, complete, both or neither? Use examples and the definitions.

What we expect: 2–3 sentences for each property (4–6 in total).

Solution. [TODO: answer]

(7b) Safety-critical deployment

System A is sound but not complete, and System B is complete but not sound. Which would you deploy in a safety-critical setting, where acting on a false conclusion has severe consequences, and why?

What we expect: 2–3 sentences tied to the definitions.

Solution. [TODO: answer]

(7c) With a separate verifier

With a separate verifier available (e.g. a human in the loop) and the goal of capturing as much true information as possible, which system would you choose, and why?

What we expect: 2–3 sentences tied to the definitions.

Solution. [TODO: answer]